STUDY PLAN IB · COMPAGNON DE L’AUDIT

Guide de prompts

Réparer le site, lot par lot

Dix-neuf textes à coller dans une session d’assistant de code, dans l’ordre. Chacun traite un groupe de constats qui partagent une cause, dit ce qu’on attend, ce qu’on ne veut pas, et comment vérifier. Rien n’est laissé à l’improvisation — surtout pas la définition de « terminé ».

OBJET	Les 121 constats de l’audit, regroupés en 19 lots
ORDRE	Sécurité → mensonges → pertes de travail → confort
CHARGE ESTIMÉE	Une dizaine de jours, hors concertation
COMPORTE	1 prompt maître, 1 gabarit, 10 règles, 19 lots, 5 contrôles
USAGE	Un lot par session. Un commit par lot. Aucun lot sans preuve.

Sommaire

Mode d’emploi . . . 3
Le prompt maître . . . 4
Le gabarit . . . 5
Les dix règles . . . 6
Les dix-neuf lots . . . 7
Lot 0 — Fermer la porte . . . 8
Lot 1 — L’onglet Examens qui ne répond pas . . . 9
Lot 2 — Le planning ne ment plus sur le temps . . . 10
Lot 3 — La page Séance montre le bon jour, et garde le décor . . . 11
Lot 4 — Supprimer sans perdre . . . 12
Lot 5 — Les exercices ont enfin un état . . . 13
Lot 6 — Voir la semaine prochaine . . . 14
Lot 7 — Lire une photo d’emploi du temps . . . 15
Lot 8 — Corriger l’emploi du temps sans le casser . . . 16
Lot 9 — Synchronisation : ne plus perdre de travail . . . 17
Lot 10 — Modifier un devoir, modifier un examen . . . 18
Lot 11 — Finir la page Devoirs . . . 19
Lot 12 — Le minuteur qui tient . . . 20
Lot 13 — Le son et la voix . . . 21
Lot 14 — Le clavier, les modales, les contrastes . . . 22
Lot 15 — Démarrage et hors-ligne . . . 23
Lot 16 — Contenu, promesses et inscription . . . 24
Lot 17 — Dette technique . . . 25
Lot 18 — Nouvelles capacités . . . 26
Les cinq contrôles . . . 27
Ce qu’il ne faut pas écrire . . . 29
Tableau de suivi . . . 30

Mode d’emploi

Ce document ne corrige rien. Il contient les textes à coller dans une session d’assistant de code pour faire appliquer l’audit, lot par lot, sans casser ce qui marche. Chaque lot correspond à un ensemble de constats qui partagent une cause : on obtient un résultat vérifiable, on coche, on passe au suivant.

> **Ce que ce document n’est pas**
> Ce n’est pas une liste de 121 ordres. Les constats qui partagent une cause sont regroupés : c’est la seule façon d’obtenir un résultat qu’on peut tester. Et ce n’est pas un mode d’emploi de l’assistant : c’est un contrat — ce qu’on demande, ce qu’on interdit, et la preuve qu’on exige en retour.

1. Ouvrez une nouvelle session

Un lot = une session. Ne mélangez pas deux lots dans la même conversation : l’assistant perd le fil, et vous ne savez plus ce qui a été fait quand quelque chose casse.

2. Collez d’abord le prompt maître

Il rappelle les règles du projet — ce qu’on ne touche pas, ce qu’on vérifie avant de livrer. Sans lui, l’assistant improvise la définition du « terminé ».

3. Collez ensuite le prompt du lot

Un seul. Lisez la ligne « Vous devez obtenir » avant de l’envoyer : c’est votre critère d’acceptation. Si vous ne savez pas la vérifier vous-même, demandez à l’assistant de vous donner la manipulation exacte.

4. Exigez la preuve

Fichiers modifiés, résultat des vérificateurs, manipulation de test. Une réponse qui dit « c’est corrigé » sans rien de tout cela n’est pas une réponse.

5. Testez vous-même sur le site déployé

Le code peut être juste et le déploiement en retard (cache, build Pages). Si le comportement n’a pas changé, attendez la fin du déploiement avant de rouvrir le lot.

6. Committez un lot à la fois

Un message de commit qui reprend l’identifiant du lot (« Lot 4 — annulation des suppressions (ET-02, ET-03, DE-05, AU-07) »). Ainsi, un retour arrière reste possible sans perdre les lots précédents.

> **Combien de temps, honnêtement**
> Un lot par session, et une session ne se termine pas toujours dans la journée. Les lots 0 à 6 tiennent en une semaine à temps plein ; le lot 9 (synchronisation) mérite une journée seule, avec une sauvegarde avant de commencer ; le lot 18 n’est pas obligatoire. Si vous ne faites que les lots 0, 1, 2, 3 et 5, vous aurez déjà réglé les cinq constats les plus visibles du site.

Le prompt maître

À coller **au début de chaque session**, avant le prompt du lot. Il rappelle les règles du projet et fixe ce que « terminé » veut dire. Sans lui, l'assistant improvise — et il improvise rarement dans le sens de vos contraintes.

PROMPT MAÎTRE — À COLLER EN PREMIER

Tu vas travailler sur Study Plan IB, un site statique (HTML, CSS, JavaScript, sans étape de build) publié sur GitHub Pages. Il sert à un élève de DP1 : emploi du temps, planning de révisions, minuteur Pomodoro, devoirs et examens. Les données vivent dans le navigateur (localStorage) et se synchronisent avec un compte Google.

Avant de répondre, lis les fichiers concernés par ma demande, ainsi que js/registre.js et js/boot.js. Ne suppose rien sur le contenu d'un fichier : ouvre-le.

Mes règles, valables pour toutes mes demandes :

1. Ne change que ce que je demande. Pas de refactor opportuniste, pas de « pendant que j'y suis », pas de reformatage. Si tu vois autre chose à corriger, signale-le à la fin, sans y toucher.
2. Avant d'écrire du code, décris en cinq lignes ce que tu vas faire et dans quels fichiers. Si cela touche plus de trois fichiers, arrête-toi et attends mon accord.
3. Après, montre-moi ce que tu as fait : les fichiers modifiés, le résumé du diff, et la preuve que ça fonctionne. Jamais « c'est corrigé » sans preuve.
4. Aucune correction ne doit notifier les utilisateurs : pas de message, pas de bannière, pas d'explication affichée. Le correctif vaut pour tout le monde, silencieusement.
5. Règles métier intouchables : jamais de nouvelle concentration automatique après le bip ; la musique s'arrête complètement quand la pause commence, puis la voix parle seule ; les fériés sont gérés par date précise ; le site ne propose jamais de réviser pendant un cours ; jamais de cours inventé.
6. La voix doit s'entendre sur un vrai navigateur. Un test hors du site ne compte pas.
7. Aucun mail de test, jamais.
8. Avant de livrer, lance node --check sur chaque fichier JavaScript modifié, puis les vérificateurs un par un (node tools/verifier-*.js). Donne le résultat exact de chacun, y compris les échecs.
9. Si un point est ambigu, ou si deux constats se contredisent, pose la question au lieu de choisir.
10. Si une partie du correctif dépend d'autre chose que du dépôt (Apps Script, console Google, réglage d'hébergement), ne l'improvise pas : liste-la séparément dans une section « à faire hors dépôt ».
11. Tu écris en français, et tu tutoies l'élève dans le code que tu produis (c'est la voix du site).

Commence par me dire ce que tu as compris de la demande, puis attends mon feu vert si la règle 2 s'applique.

Les règles 1 à 3 sont celles qui font la différence. Un assistant qui n'a pas l'interdiction explicite du « pendant que j'y suis » ajoutera des changements que vous ne pourrez pas relire. Et un assistant à qui on ne demande pas de preuve vous répondra que tout fonctionne.

Le gabarit

Les prompts des pages suivantes suivent tous cette structure. Vous pouvez l’utiliser pour écrire les vôtres — pour un bug que vous découvrez, ou pour un constat qui mérite d’être traité différemment.

GABARIT — À ADAPTER

```
Lot : <numéro et nom>
Constat(s) de l'audit : <identifiants, par exemple DE-01, DE-02>
But : en une phrase, ce que l'élève doit pouvoir faire à la fin.
Fichiers concernés : <liste des fichiers à ouvrir>

Ce que je veux exactement :
1. <le comportement attendu, formulé comme une phrase de test>
2. <le cas limite à traiter>
3. <ce qui doit rester inchangé>

Ce que je ne veux pas : <les tentations à écarter pour ce lot>

Comment on vérifie : <la manipulation précise, celle que je peux refaire moi-même>

Réponds dans cet ordre : le plan, le code, la preuve. Pas de conclusion avant la preuve.
```

Pourquoi ces six lignes

Le but en une phrase

C’est ce qui empêche l’assistant de choisir une autre interprétation que la vôtre. « L’import des examens doit afficher ses résultats » ne laisse pas de place à une refonte de la page.

Les fichiers

Un assistant qui doit chercher lui-même ouvre vingt fichiers et en modifie trois de trop. Nommez-les.

Ce que je veux exactement

Numéroté, formulé comme un test. Chaque point doit pouvoir se vérifier par une manipulation.

Ce que je ne veux pas

La ligne la plus utile, et celle que tout le monde oublie. C’est elle qui évite le refactor opportuniste.

Comment on vérifie

Si vous ne pouvez pas l’écrire, le correctif n’est pas défini. C’est un critère, pas une formalité.

Le format de réponse

Le plan d’abord, la preuve ensuite. Cela vous laisse arrêter avant que trois cents lignes soient écrites dans la mauvaise direction.

Les dix règles

Elles ne dépendent d’aucun lot. Elles valent pour toutes les demandes, y compris celles que vous écrirez vous-même.

1. Un lot à la fois
Deux lots dans une session, et vous ne saurez plus lequel a cassé quoi. L’assistant, lui, mélange les fichiers et perd la moitié des contraintes.

2. Un constat = un test
Pour chaque problème corrigé, il doit exister une manipulation qui échoue avant et réussit après. Si vous ne pouvez pas l’écrire, le correctif n’est pas vérifiable.

3. Toujours demander la preuve
« Montre-moi le diff », « donne-moi le résultat de node --check », « quelle est la commande qui prouve que ça marche ? ». C’est ce qui distingue un correctif d’une intention.

4. La sécurité d’abord
Les lots 0 à 2 ne sont pas négociables et ne s’améliorent pas avec le temps : chaque semaine d’attente ajoute des données exposées. Le reste peut attendre.

5. Ne jamais demander « corrige tout »
Un prompt qui demande les 121 constats d’un coup produit cent vingt et un changements dont aucun n’est testé. C’est la garantie d’un site cassé et d’un retour arrière complet.

6. Un commit par lot
Le message reprend les identifiants. Si un lot casse quelque chose deux semaines plus tard, vous pouvez l’annuler seul.

7. Relire avant d’accepter
Demandez toujours un second passage : « relis ton propre diff et cherche ce que tu as cassé ». Un assistant trouve ses propres erreurs une fois sur deux.

8. Ne pas mélanger correctif et design
Si un lot change l’apparence en plus du comportement, vous ne saurez pas ce qui a cassé l’affichage. Le design se demande séparément.

9. Vérifier sur le site déployé
Le code peut être juste et le déploiement en retard. Si le comportement n’a pas changé, attendez la fin du déploiement avant de conclure.

10. Tenir le décompte
Le tableau de suivi à la fin du document sert à cela. Ce qui n’est pas coché n’est pas fait, même si l’assistant a dit qu’il l’avait fait.

Les dix-neuf lots

Dans l’ordre. Les numéros ne sont pas décoratifs : chaque lot suppose les précédents faits, sauf mention contraire. La colonne « quand » le dit. Les 121 constats de l’audit sont répartis dans les dix-neuf lots, chacun dans un seul — c’est vérifié à la construction de ce document.

Lot	Ce qu’il règle	Constats	Effort
0	Fermer la porte	5 constats	6 h
1	L’onglet Examens qui ne répond pas	4 constats	1 h 30
2	Le planning ne ment plus sur le temps	4 constats	3 h 30
3	La page Séance montre le bon jour, et garde le décor	2 constats	1 h 30
4	Supprimer sans perdre	5 constats	2 h 30
5	Les exercices ont enfin un état	2 constats	1 h 30
6	Voir la semaine prochaine	2 constats	1 j
7	Lire une photo d’emploi du temps	5 constats	5 h
8	Corriger l’emploi du temps sans le casser	8 constats	6 h
9	Synchronisation : ne plus perdre de travail	6 constats	8 h
10	Modifier un devoir, modifier un examen	4 constats	4 h 30
11	Finir la page Devoirs	12 constats	5 h
12	Le minuteur qui tient	6 constats	5 h
13	Le son et la voix	11 constats	6 h
14	Le clavier, les modales, les contrastes	11 constats	8 h
15	Démarrage et hors-ligne	6 constats	1 j 1/2
16	Contenu, promesses et inscription	19 constats	1 j 1/2
17	Dette technique	5 constats	3 j
18	Nouvelles capacités	4 constats	3 à 5 j

Total : 121 constats couverts, et quelques ajouts du lot 18 qui ne sont pas des corrections. Les lots 0 à 3 représentent à eux seuls la moitié du bénéfice perçu pour un quart du travail.

Le chemin court

Si vous ne faites que cinq lots, faites ceux-là, dans cet ordre : **0** (la porte), **1** (les examens qui ne répondent pas), **2** (les révisions dans le passé), **3** (la page Séance), **5** (les exercices qui ne se terminent jamais). Onze heures au total, et le site change de visage.

Le reste peut attendre. Ce qui ne peut pas attendre, c’est le lot 0 : chaque semaine d’attente ajoute des messages exposés sur un canal public.

Lot 0 — Fermer la porte

Vague 0 · 6 h

CONSTATS SE-01, SE-02, SE-03, SE-04, SE-05

FICHIERS js/pages/boite.js, data/boite-url.json, index.html

QUAND Aujourd’hui. Rien d’autre ne passe avant.

PROMPT DU LOT 0 — À COLLER APRÈS LE PROMPT MAÎTRE

Lot 0 — Sécurité de la boîte de messages.

Constat(s) : SE-01, SE-02, SE-03, SE-04, SE-05.

But : qu'aucun élève ne puisse lire la boîte des responsables en tapant un prénom, et que le secret d'envoi de mails ne soit plus dans une page publique.

Fais, dans cet ordre :

1. Remplace le contrôle d'administration par le nom (BOITE_ADMINS, dans js/pages/boite.js) par un contrôle sur le sub Google de la session (BOITE_ADMINS_SUB). Si la session n'est pas vérifiée, l'accès admin est refusé. Retire les noms complets et les adresses e-mail du code livré.

2. Sors le nom du fil ntfy du code : sers-le par data/boite-url.json, déjà utilisé pour l'URL du script, et régénère ce nom. Explique-moi comment faire la régénération côté service.

3. Supprime BOITE_SCRIPT_SECRET du client. Fais vérifier l'identité par le jeton de session côté Apps Script. Donne-moi le code Apps Script à coller, séparément, dans une section « à faire hors dépôt ».

4. À la déconnexion, propose « Effacer mes données de cet appareil ». Au retour, si le profil local ne correspond pas au compte Google actif, demande explicitement « Continuer en tant que X » ou « Ce n'est pas moi ».

5. Ajoute une politique de sécurité de contenu en mode Report-Only, et l'attribut integrity sur chaque ressource externe.

Ce que je ne veux pas : changer l'apparence de la boîte, toucher au fonctionnement pour les quatre responsables légitimes, ajouter une dépendance.

Vous devez obtenir

— Le code source de la page ne contient plus de nom de fil ntfy, ni de secret, ni de nom de personne.

— Un profil non-admin qui tape un prénom de responsable n’obtient aucun accès.

— Les quatre responsables accèdent toujours, avec leur compte Google.

— La déconnexion propose d’effacer les données locales.

Le piège de ce lot

Ne pas laisser une « solution de repli » qui réautorise l’accès par le prénom quand la session échoue. C’est exactement le trou qu’on ferme.

Lot 1 — L’onglet Examens qui ne répond pas

Vague 1 · 1 h 30

CONSTATS DE-01, DE-02, DE-21, DE-22

FICHIERS pages/devoirs.html, js/pages/devoirs.js

QUAND Le lendemain du lot 0.

PROMPT DU LOT 1 — À COLLER APRÈS LE PROMPT MAÎTRE

Lot 1 — Rendre visible l'import des examens.

Constat(s) : DE-01, DE-02, DE-21, DE-22.

But : quand je photographie ou colle un planning d'examens dans l'onglet Examens, je dois voir les lignes détectées, sous mes yeux, et savoir ce qui s'est passé.

Fais :

1. Il n'existe qu'une zone de confirmation (#devoirsConfirmation), dans l'onglet Devoirs. Rends la zone commune : sors-la des onglets pour qu'elle soit visible quel que soit l'onglet actif, plutôt que d'en dupliquer une seconde.

2. Fais passer le genre à la fonction qui écrit le statut, au lieu de le déduire de l'onglet actif au moment de la confirmation. Une seule fonction doit résoudre la bonne zone à partir du genre.

3. Ajoute onchange="devoirsLireDocument()" sur l'input de fichier de l'onglet Examens, qui n'en a aucun. Vérifie que la lecture sait de quel onglet vient le fichier.

4. Ajoute aria-live="polite" et role="status" sur les trois zones de statut (devoirsOcrStatut, exaOcrStatut, devoirsAjoutStatut), et annonce un résumé après chaque action : nombre de créneaux détectés, nombre ajoutés, ou raison de l'échec.

Ce que je ne veux pas : déplacer les onglets, changer la mise en page du reste de la page, modifier le parseur.

Vous devez obtenir

— Une seule zone de confirmation, visible depuis les deux onglets.

— Le statut se met à jour dans le bon onglet, quel que soit celui qui est actif.

— Choisir un fichier dans l’onglet Examens déclenche la lecture.

— Le résultat est annoncé au lecteur d’écran.

Le piège de ce lot

Ne pas dupliquer la zone : deux zones, c’est deux états à garder synchronisés, et le bug revient par l’autre côté.

Lot 2 — Le planning ne ment plus sur le temps

Vague 1 · 3 h 30

CONSTATS DE-14, DE-15, DE-16, DE-19

FICHIERS js/pages/devoirs-moteur.js, js/pages/emploi-temps-moteur.js, js/pages/devoirs.js

QUAND Après le lot 1.

PROMPT DU LOT 2 — À COLLER APRÈS LE PROMPT MAÎTRE

Lot 2 — Les révisions doivent tomber sur des jours qui existent encore.

Constat(s) : DE-14, DE-15, DE-16, DE-19.

But : aucune révision dans le passé, aucune révision hors sujet pendant la semaine d'examen, et pas de revision d'un examen lointain dès le lendemain.

Fais :

1. Dans devoirsAdapterJour, ajoute une garde unique : si le jour traité est antérieur à la date du jour, ne pose aucun bloc de révision. Les blocs fixes (composition, examen du jour) peuvent rester, ils ont eu lieu.

2. Dans devoirsBlocARetirer, les blocs d'étude issus d'un emploi du temps chargé portent les identifiants emploi-rev-<heure> et emploi-exo-<heure> : ils ne sont pas retirés aujourd'hui. Traite ces deux préfixes. Si tu peux, remplace la décision par une propriété explicite portée par le bloc plutôt que par des préfixes d'identifiant — mais ne casse pas les blocs existants.

3. N'ouvre les séances d'un examen qu'à partir de J-14 avant l'épreuve (J-21 si tu estimes que c'est nécessaire pour un examen majeur, dis-le-moi avant). Le nombre de séances se calcule dans cette fenêtre, pas depuis aujourd'hui.

4. Dans devoirsParserTexte, si aucune date ne correspond après la boucle de recherche, ne renvoie pas « aujourd'hui + 60 jours » : renvoie une date vide et marque la ligne « date à préciser ».

Ce que je ne veux pas : changer le nombre de séances par matière, toucher aux règles de répartition déjà validées (au moins une la veille, jamais le jour du devoir), modifier l'affichage du planning.

Vous devez obtenir

— Aucune révision dans un jour antérieur à aujourd’hui.

— Pendant la semaine d’examen, il ne reste que les révisions des matières examinées.

— Un examen lointain ne remplit pas les semaines proches.

— Une date illisible est signalée, pas inventée.

Le piège de ce lot

Le point 2 casse facilement les blocs de mémoire et d’IA. Vérifie qu’ils sont toujours retirés, eux aussi.

Lot 3 — La page Séance montre le bon jour, et garde le décor

Vague 1 · 1 h 30

CONSTATS PO-01, PO-02

FICHIERS js/pages/pomodoro-page.js, js/pages/planning-base.js

QUAND Après le lot 2.

PROMPT DU LOT 3 — À COLLER APRÈS LE PROMPT MAÎTRE

Lot 3 — La page Séance doit parler du jour qu'elle affiche, et garder mes choix.

Constat(s) : PO-01, PO-02.

But : le titre de la page et la liste des séances doivent toujours parler du même jour ; et le décor que je choisis doit être là quand je reviens.

Fais :

1. focusJourIndex() retourne selectedDay, la variable du planning, que selectDay() modifie à chaque clic sur un jour. Découple-les : la page Séance utilise le jour courant (indexJourCourant). Si tu veux permettre de consulter un autre jour, introduis un état local propre à la page, avec un sélecteur explicite et un titre qui suit.

2. focusDateTexte() doit afficher la date du jour réellement affiché, pas la date du système.

3. Le décor : focusPoserFond() écrit studyPlanIB_focusFond mais personne ne la relit, et focusOuvrirSeances() comme focusOuvrirMinuteur() rappellent focusPoserFond avec le décor par défaut. Lis la clé au chargement, avec repli sur lumiere uniquement si elle est absente ou invalide. Supprime le forçage des deux ouvertures.

Ce que je ne veux pas : changer le rendu visuel du décor, toucher au chrono, modifier le comportement du planning quand on clique sur un jour.

Vous devez obtenir

— Titre et liste des séances toujours cohérents.

— Le décor survit à un rechargement.

— Le décor par défaut revient seulement si la préférence n’existe pas.

— Le planning garde son comportement de sélection de jour.

Le piège de ce lot

Tester aussi après un changement de page (planning → séance → planning), pas seulement après un rechargement.

Lot 4 — Supprimer sans perdre

Vague 1 · 2 h 30

CONSTATS ET-02, ET-03, DE-05, AU-07, AU-10

FICHIERS js/pages/emploi-temps.js, js/pages/devoirs.js, js/pages/exercices.js

QUAND Après le lot 3.

PROMPT DU LOT 4 — À COLLER APRÈS LE PROMPT MAÎTRE

Lot 4 — Une seule façon d'annuler une suppression, partout.

Constat(s) : ET-02, ET-03, DE-05, AU-07, AU-10.

But : supprimer un cours, un devoir ou un exercice doit pouvoir être annulé pendant quelques secondes, et la suppression d'un cours doit m'avertir si elle emporte des révisions.

Fais :

1. Crée une primitive unique d'annulation : suppression optimiste, toast pendant 8 secondes avec un bouton « Annuler », restauration complète de l'objet s'il est cliqué. Un seul endroit dans le code, utilisé par les quatre suppressions.

2. Dans l'emploi du temps, avant de supprimer un cours, compte les blocs de révision liés et annonce-le dans le toast : « Ce cours porte 3 séances de révision — supprimer aussi ? ». Si les heures creuses existent encore, propose de les déplacer à la place.

3. Sort « tout effacer » de la phrase qui annonce le nombre de cours : bouton secondaire, à droite du titre de la liste. Il ouvre une confirmation qui nomme l'objet et le volume, et propose de télécharger les données d'abord.

4. Les trois fonctions de suppression doivent régénérer le planning après coup (c'est DE-06, à faire ici puisqu'on touche le même chemin).

5. Remplace les quatre confirm() du site (effacer la progression, remettre le planning du jour, remplacer le Pomodoro, changer de rythme) par la modale interne, avec le nom de l'objet concerné dans le titre. Deux de ces quatre appels sont dans un module mort : supprime-les avec lui.

Ce que je ne veux pas : un système de corbeille avec une page dédiée, une durée d'annulation configurable, une confirmation supplémentaire sur chaque suppression simple.

Vous devez obtenir

— Une seule implémentation, réutilisée trois fois.

— La restauration remet aussi les blocs de révision liés.

— « Tout effacer » demande confirmation et nomme le volume.

— Le planning est régénéré après chaque suppression.

— Plus aucun confirm() du navigateur sur les actions destructrices.

Le piège de ce lot

Si le toast est reconstruit à chaque rendu, l’annulation saute. La primitive doit vivre en dehors de la fonction de rendu.

Lot 5 — Les exercices ont enfin un état

Vague 1 · 1 h 30

CONSTATS AU-06, AU-09

FICHIERS js/pages/exercices.js, pages/exercices.html

QUAND Après le lot 4.

PROMPT DU LOT 5 — À COLLER APRÈS LE PROMPT MAÎTRE

Lot 5 — Je dois pouvoir marquer un exercice comme fait.

Constat(s) : AU-06, AU-09.

But : quand j'ai fait un exercice, le site doit le savoir, et le compteur doit descendre.

Fais :

1. Ajoute un bouton « Fait » sur chaque carte d'exercice, qui passe done à true, écrit le stockage et met à jour les trois compteurs (restants, minutes restantes, liste à planifier). Le champ done existe déjà et n'est jamais passé à true : vérifie-le avant d'ajouter quoi que ce soit.

2. Prévois l'inverse (décocher), et un filtre « à faire / faits ».

3. Remplace les trois alert() de validation par des messages en ligne sous le champ concerné, avec aria-live, et place le focus sur le champ fautif.

Ce que je ne veux pas : supprimer l'exercice coché automatiquement, changer la structure de la carte, modifier le calcul de difficulté.

Vous devez obtenir

— Le compteur descend réellement.

— L’état survit au rechargement.

— L’inverse fonctionne.

— Plus aucune alerte bloquante dans cette page.

Le piège de ce lot

Les trois filtres lisent done à des endroits différents. Vérifie-les un par un, sinon le compteur descend mais la liste ne bouge pas.

Lot 6 — Voir la semaine prochaine

Vague 1 · 1 j

CONSTATS AU-01, PO-19

FICHIERS js/pages/planning-base.js, js/pages/planning-interface.js, pages/planning.html

QUAND Après le lot 5.

PROMPT DU LOT 6 — À COLLER APRÈS LE PROMPT MAÎTRE

Lot 6 — Je dois pouvoir regarder la semaine prochaine.

Constat(s) : AU-01, PO-19.

But : le planning doit permettre d'avancer et de reculer d'une semaine, sans casser les révisions que le moteur y place.

Fais :

1. getWeekDates() calcule la semaine à partir d'aujourd'hui et n'accepte aucun décalage. Ajoute un paramètre de décalage, et un état de navigation dans l'interface.

2. Deux flèches dans l'en-tête, avec les dates affichées (« 20 – 26 oct. ») et un retour rapide « Aujourd'hui ». Quand on regarde une autre semaine, l'afficher clairement.

3. Le moteur reçoit déjà un index de jour : assure-toi que les dates passées au moteur suivent la semaine affichée, et que devoirsDateDuJour suit aussi. Les révisions doivent se poser sur les bonnes dates, pas sur celles de la semaine réelle.

4. La page Séance ne doit pas hériter de ce décalage (c'est le lot 3, vérifie simplement que rien n'a régressé).

5. Ajoute un indicateur discret dans l'en-tête quand un minuteur tourne encore : « 12:34 », cliquable pour revenir à la page Pomodoro.

Ce que je ne veux pas : changer la structure de la grille, ajouter un calendrier mensuel, déplacer les boutons existants.

Vous devez obtenir

— Navigation avant / arrière, avec dates visibles.

— Les révisions se placent sur la semaine affichée.

— Retour « Aujourd’hui » immédiat.

— La page Séance n’est pas affectée.

Le piège de ce lot

Si le moteur mémorise ses blocs par date réelle, changer la semaine peut dupliquer les révisions. Vérifie deux semaines d’affilée.

Lot 7 — Lire une photo d’emploi du temps

Vague 2 · 5 h

CONSTATS ET-05, ET-06, ET-07, ET-08, ET-15

FICHIERS js/pages/emploi-temps.js, pages/emploi-temps.html

QUAND Après la vague 1 complète.

PROMPT DU LOT 7 — À COLLER APRÈS LE PROMPT MAÎTRE

Lot 7 — L'import d'un emploi du temps doit produire un emploi du temps juste.

Constat(s) : ET-05, ET-06, ET-07, ET-08, ET-15.

But : je photographie mon emploi du temps, et les cours se placent sur les bons jours, samedi compris.

Fais :

1. Le parseur lit ligne à ligne et ne reconnaît que les heures. Ajoute la lecture des en-têtes de jour (« Lundi », « Mardi 14 », « lun. » ...) : une ligne d'en-tête change le jour courant et n'est jamais une matière. Une ligne sans heure et sans matière connue est ignorée.

2. L'index de jour est plafonné à 4 (lundi-vendredi). Remplace la borne par sept jours, et accepte 0 à 6. Le formulaire manuel accepte déjà le samedi : les deux chemins doivent faire la même chose.

3. Confirmer un deuxième document efface tous les cours marqués src === 'doc'. Marque chaque import avec une empreinte (par exemple doc:<date du jour>) et ne retire que les cours de la même empreinte. Par défaut, ajoute à la suite ; ne remplace que si je le demande explicitement.

4. Avant d'écrire, fais tourner un contrôle de chevauchement sur l'ensemble des propositions, et affiche en tête : « 2 conflits horaires — les vérifier », avec un fond distinct sur les lignes concernées.

5. Ajoute une progression réelle pendant la lecture (pourcentage de Tesseract, numéro de page), un bouton d'annulation, et annonce la limite des trois pages.

Ce que je ne veux pas : changer le moteur de lecture OCR lui-même, toucher à l'affichage du planning, ajouter une bibliothèque.

Vous devez obtenir

— Les en-têtes de jour pilotent le placement.

— Le samedi est possible.

— Un second import ne détruit pas le premier.

— Les conflits sont signalés avant l’écriture.

Le piège de ce lot

Le parseur est aussi utilisé pour les devoirs. Vérifie que tes changements ne dégradent pas devoirsParserTexte.

Lot 8 — Corriger l’emploi du temps sans le casser

Vague 2 · 6 h

CONSTATS ET-01, ET-04, ET-11, ET-12, ET-13, ET-14, ET-17, ET-18

FICHIERS js/pages/emploi-temps.js, pages/emploi-temps.html, js/pages/planning-moteur.js

QUAND Après le lot 7.

PROMPT DU LOT 8 — À COLLER APRÈS LE PROMPT MAÎTRE

Lot 8 — Modifier un cours doit être simple, et sans effet de bord.

Constat(s) : ET-01, ET-04, ET-11, ET-12, ET-13, ET-14, ET-17, ET-18.

But : corriger une faute de frappe ne doit plus obliger à supprimer et ressaisir, et ne doit plus décaler les révisions des autres cours.

Fais :

1. Un mode édition : un crayon sur chaque ligne, qui pré-remplit le formulaire existant, avec le même contrôle qu'à l'ajout. L'identifiant du cours ne change pas, donc les révisions survivent.

2. À l'ajout et à l'édition, contrôle de chevauchement : compare la plage aux autres cours du même jour. En cas de conflit, propose Remplacer / Garder les deux / Annuler, en nommant le cours concerné. Prévois un champ parallele pour les cours volontairement simultanés.

3. Les identifiants de cours générés dans le planning utilisent la position (cours-reel-<index>). Utilise l'identifiant stable du store (c.id), pas l'index.

4. La suppression transmet l'index de jour calculé au rendu. Transmets seulement l'identifiant du cours, et laisse la fonction le retrouver dans les sept jours.

5. Étiquettes sur les quatre champs du formulaire, aria-live sur les zones de statut, et transformation de la zone de dépôt en bouton ou en label accessible au clavier.

6. mémoïse la lecture du store et ne génère que le jour demandé quand le cache d'un jour est vide, au lieu de reconstruire sept jours à chaque affichage.

7. Renomme le bouton du bas « Voir mon planning » et laisse « Ajouter le cours » au formulaire.

Ce que je ne veux pas : changer le modèle de données existant, supprimer les cours manuels, modifier les règles du moteur de planning.

Vous devez obtenir

— Édition sans perte des révisions liées.

— Conflits détectés à l’ajout et à l’édition.

— Identifiants stables.

— Formulaire accessible au clavier et au lecteur d’écran.

Le piège de ce lot

Le point 3 touche les identifiants déjà stockés dans les plannings existants. Prévois une migration ou une lecture tolérante, sinon les anciennes révisions deviennent orphelines.

Lot 9 — Synchronisation : ne plus perdre de travail

Vague 2 · 8 h

CONSTATS SY-01, SY-02, SY-03, SY-04, DE-25, ET-16

FICHIERS js/compte.js, js/memoire.js, js/pages/emploi-temps.js

QUAND Lot le plus délicat. À faire seul, avec du temps.

PROMPT DU LOT 9 — À COLLER APRÈS LE PROMPT MAÎTRE

Lot 9 — La synchronisation doit arrêter de perdre des données.

Constat(s) : SY-01, SY-02, SY-03, SY-04, DE-25, ET-16.

But : ce que je supprime reste supprimé, ce que je modifie sur un appareil gagne sur l'autre, et l'emploi du temps du compte ne remplace plus celui de l'appareil.

Fais, dans cet ordre — et arrête-toi après le point 1 si le test est concluant, pour qu'on valide avant d'aller plus loin :

1. Pierres tombales : au lieu de retirer un objet du tableau, écris-le avec supprime à true et supprimeAt. La fusion garde la version la plus récente, et un objet supprimé ne réapparaît jamais. Purge après 60 jours.

2. modifieAt par objet, écrit à chaque modification, et comparaison stricte à la fusion : la date la plus récente gagne, à égalité la version locale. Les objets existants démarrent à 0.

3. L'emploi du temps est écrasé en bloc à la restauration (memoireAppliquerEtat). Fais-le passer par la même fusion que les devoirs, par identifiant de cours.

4. Supprime le champ selectedDay du profil : il est écrit et jamais relu.

5. Ajoute un objet focusReglages (durées, volumes, son, fond, voix) au profil synchronisé.

Ce que je ne veux pas : changer le format de sauvegarde d'une façon qui rende illisibles les sauvegardes existantes, supprimer la compatibilité ascendante, toucher au moteur de planning.

Vous devez obtenir

— Une suppression ne revient jamais.

— La modification la plus récente gagne, par objet.

— L’emploi du temps se fusionne au lieu d’être écrasé.

— Les sauvegardes existantes restent lisibles.

Le piège de ce lot

Si les tombstones ne sont pas purgées, le profil grossit sans fin et finit par dépasser le quota (AR-03). Prévois la purge dès le premier jour.

Lot 10 — Modifier un devoir, modifier un examen

Vague 2 · 4 h 30

CONSTATS DE-03, DE-04, DE-06, DE-07

FICHIERS js/pages/devoirs.js, pages/devoirs.html

QUAND Après le lot 9.

PROMPT DU LOT 10 — À COLLER APRÈS LE PROMPT MAÎTRE

Lot 10 — Corriger un devoir doit être possible.

Constat(s) : DE-03, DE-04, DE-06, DE-07.

But : une faute de date doit se corriger en un geste, sans supprimer et ressaisir.

Fais :

1. Une modale d'édition unique, ouverte par un crayon sur chaque ligne, avec les quatre champs (matière, date, début, fin) et le nombre de séances. Elle sert aux devoirs et aux examens.

2. Extrais la validation existante dans une fonction devoirsValider(champs), appelée par l'ajout et par la modification : fin après début, date cohérente, matière non vide.

3. À l'enregistrement, régénère le planning. Vérifie au passage que la suppression le fait aussi : elle ne le fait pas aujourd'hui, et c'est DE-06.

4. Avant d'ajouter une proposition issue d'un document, cherche un devoir existant de même matière et de même date. Si tu en trouves un, propose « Mettre à jour » au lieu d'« Ajouter ».

Ce que je ne veux pas : une page d'édition séparée, la perte du nombre de séances déjà réglé, un changement de l'affichage de la liste.

Vous devez obtenir

— Édition possible sur un devoir et sur un examen.

— Le planning est régénéré après modification et après suppression.

— Un second import propose la mise à jour, pas le doublon.

Le piège de ce lot

Modifier la date d’un examen change toute la semaine d’examen. Vérifie qu’aucun ancien bloc ne reste orphelin.

Lot 11 — Finir la page Devoirs

Vague 2 · 5 h

CONSTATS DE-08, DE-09, DE-10, DE-11, DE-12, DE-13, DE-17, DE-18, DE-20, DE-23, DE-24, DE-26

FICHIERS js/pages/devoirs.js, js/pages/devoirs-moteur.js, pages/devoirs.html

QUAND Après le lot 10.

PROMPT DU LOT 11 — À COLLER APRÈS LE PROMPT MAÎTRE

Lot 11 — Les finitions de la page Devoirs.

Constat(s) : DE-08, DE-09, DE-10, DE-11, DE-12, DE-13, DE-17, DE-18, DE-20, DE-23, DE-24, DE-26.

But : la liste ne doit plus commencer par des lignes mortes, et je dois pouvoir choisir zéro séance.

Fais :

1. Sépare la liste en « À venir » et « Passés » (repliés, avec le nombre). Même chose pour les examens. Marque les examens terminés et ne génère plus de révisions pour eux.

2. Remplace le prompt() du nombre de séances par la modale interne, à trois issues : Annuler, Aucune séance, valeur. Traite null comme une annulation. Autorise 0 séance de façon explicite.

3. Il existe deux plafonds contradictoires (6 et 10) dans deux fonctions. Une seule constante, utilisée partout.

4. Contrôle de chevauchement à l'ajout d'un devoir ou d'un examen, avec proposition de corriger ou de garder les deux. En affichage, juxtapose les blocs plutôt que de les empiler.

5. Un devoir sans heures et sans créneau « devoir » n'apparaît jamais : pose-le à un horaire par défaut marqué « heure à confirmer », et signale-le dans la liste.

6. Corrige la conversion des heures anglo-saxonnes : 12 am vaut 0 h, 12 pm vaut 12 h.

7. Étiquettes sur les champs, champ de recherche, filtres à venir/passés, et affichage du nombre de séances et du compte à rebours sur chaque ligne.

8. Les fenêtres de cette page sont des div superposés, sans role dialog, sans piège de focus, sans fermeture par Échap. Applique-leur le traitement modal (c'est le sujet du lot 14, mais commence ici par ces deux fenêtres).

Ce que je ne veux pas : refondre la page, changer le moteur de répartition, supprimer le mode manuel.

Vous devez obtenir

— Liste séparée à venir / passés.

— Zéro séance possible.

— Un seul plafond de séances.

— Un devoir sans heure reste visible.

— Les fenêtres de la page retiennent le focus et se ferment par Échap.

Le piège de ce lot

Le point 5 ajoute des blocs à 8h par défaut : vérifie qu’ils ne se superposent pas aux cours du matin (DE-17).

Lot 12 — Le minuteur qui tient

Vague 3 · 5 h

CONSTATS PO-03, PO-04, PO-05, PO-11, PO-12, PO-20

FICHIERS js/pages/pomodoro-page.js, js/pages/pomodoro.js, js/registre.js

QUAND Après la vague 2.

PROMPT DU LOT 12 — À COLLER APRÈS LE PROMPT MAÎTRE

Lot 12 — Le minuteur doit survivre à un rechargement, et ne compter qu'une fois.

Constat(s) : PO-03, PO-04, PO-05, PO-11, PO-12, PO-20.

But : je lance une séance, je verrouille mon téléphone, je reviens : le chrono est au bon endroit.

Fais :

1. Persiste un état minimal (mode, total, restant, running, cycles, date) et restaure-le au chargement. Si le chrono tournait, recalcule le restant à partir de l'heure de fin prévue, pas à partir de l'intervalle. Si la date n'est pas le jour même, repars proprement.

2. Inclus les cycles dans cet état, remis à zéro à minuit et non à chaque rechargement. Affiche le cycle en cours (« cycle 2 / 4 »).

3. Le module pomodoro.js est mort mais chargé, et contient un enchaînement automatique concentration/pause, ce que le site s'interdit. Supprime-le et retire son entrée du registre, en déplaçant les deux fonctions utilitaires encore utilisées dans pomodoro-page.js.

4. Une seule source de durées : focusDurees(). Supprime le preset 25/5 et le libellé « × 25 min » écrit en dur, ainsi que le texte fixe « sessions de 25 min » de l'accueil.

5. Sépare le rendu : le chrono se redessine toutes les 250 ms, le reste seulement au changement d'état. Le titre de l'onglet se met à jour une fois par minute.

6. Changer une durée ne doit plus remettre à zéro un autre mode en cours.

Ce que je ne veux pas : de nouvelle concentration automatique après le bip, un changement de la logique de pause longue, une modification du son.

Vous devez obtenir

— Le chrono survit au rechargement.

— Les cycles survivent et la pause longue arrive.

— Un seul jeu de durées dans toute l’interface.

— Le module mort n’est plus chargé.

Le piège de ce lot

La restauration doit être silencieuse : pas de bannière « session restaurée » (règle 4 du prompt maître).

Lot 13 — Le son et la voix

Vague 3 · 6 h

CONSTATS	PO-06, PO-07, PO-08, PO-09, PO-10, PO-13, PO-14, PO-15, PO-16, PO-17, PO-18
FICHIERS	js/pages/pomodoro-page.js, js/pages/chrono-voix.js, js/pages/musique.js, audio/voix/
QUAND	Après le lot 12. Nécessite un vrai navigateur.

PROMPT DU LOT 13 — À COLLER APRÈS LE PROMPT MAÎTRE

Lot 13 — Le son doit se comporter comme promis.

Constat(s) : PO-06, PO-07, PO-08, PO-09, PO-10, PO-13, PO-14, PO-15, PO-16, PO-17, PO-18.

But : la musique s'arrête complètement quand la pause commence, la voix parle seule, et je peux couper la voix si je veux.

Fais :

1. Si je relance la pause pendant l'annonce, la musique revient par-dessus la voix. Pose un drapeau pendant l'annonce : toute reprise de musique est mise en attente et jouée à la fin, ou abandonnée si une autre phase a commencé.

2. Ajoute un réglage voix (activée / coupée) et un volume, indépendants du bip. Mémorise-les.

3. Mémorise le son d'ambiance choisi, sans jamais le démarrer automatiquement au chargement (les navigateurs l'interdisent sans geste).

4. Retire les cinq écouteurs qui bloquent copie, coupe, sélection, clic droit et Ctrl+C sur toute la page. S'ils protégeaient un élément précis, cible cet élément uniquement.

5. Le raccourci Espace ne doit pas se déclencher quand le focus est sur un bouton, un champ ou une zone de texte.

6. Lancer une séance ne doit plus écraser un objectif déjà saisi : propose de le conserver ou de l'accoler au titre de la séance.

7. Ajoute un contenu de repli dans le HTML des pages minuteur, et un message si le script échoue.

8. aria-pressed sur les boutons de mode, et une annonce du changement de mode.

9. Le fichier pause-fin.wav n'existe pas : la pause se termine en silence. Prépare le code pour qu'il le lise s'il est présent, et donne-moi la phrase exacte à enregistrer, au tutoiement, sur le modèle des fichiers existants. Ne fabrique pas le son toi-même.

Ce que je ne veux pas : de nouvelle concentration automatique, un changement du moment où le bip retentit, une nouvelle bibliothèque audio.

Vous devez obtenir

— Aucune superposition musique / voix.

— Interrupteur de voix et volume mémorisés.

— Ambiance mémorisée, jamais lancée seule.

— Copier-coller de nouveau possible.

Le piège de ce lot

Ne touche pas au moment du bip ni à l’ordre musique → bip → voix : c’est le seul comportement validé sur appareil réel.

Lot 14 — Le clavier, les modales, les contrastes

Vague 3 · 8 h

CONSTATS AC-01, AC-02, AC-03, AC-04, AC-05, AC-06, AC-07, AC-09, AC-10, AC-11, AU-12

FICHIERS index.html, css/, pages/, js/pages/

QUAND Après le lot 13.

PROMPT DU LOT 14 — À COLLER APRÈS LE PROMPT MAÎTRE

Lot 14 — Accessibilité de base, sur tout le site.

Constat(s) : AC-01, AC-02, AC-03, AC-04, AC-05, AC-06, AC-07, AC-09, AC-10, AC-11, AU-12.

But : le site doit être utilisable au clavier et au lecteur d'écran, sans refonte visuelle.

Fais :

1. Une région d'annonce unique et persistante dans le gabarit, et une fonction annoncer(texte) utilisée par toutes les actions qui modifient quelque chose. C'est le point le plus structurant.

2. Une règle globale :focus-visible visible, et supprime les 32 outline:none (ou remplace-les par un style visible).

3. Migre les quinze modales sur un composant unique : role dialog, aria-modal, piège de focus, Échap, restitution du focus à la fermeture.

4. aria-label sur les dix-sept boutons qui n'ont qu'un pictogramme.

5. Étiquettes associées sur tous les champs (visibles ou sr-only).

6. Remplace les gris trop clairs par une valeur conforme pour le texte courant ; garde le gris clair pour le décoratif seulement.

7. Respecte prefers-reduced-motion, et ajoute les dimensions sur les cinq images sans width/height.

8. Corrige le violet partagé entre Activités et Mémoire : deux couleurs distinctes, et un motif en plus de la couleur.

9. Corrige les débordements masqués : les largeurs fixes inférieures à 400 px passent en min(100 %, ...), les rangées en flex-wrap, et les overflow hidden qui cachent du contenu disparaissent. Teste à 320 px de large.

10. Étiquette sur les champs de recherche qui n'ont qu'un placeholder.

Ce que je ne veux pas : changer la charte graphique, réécrire les feuilles de style, modifier la structure des pages au-delà du strict nécessaire.

Vous devez obtenir

— Une région d’annonce et une fonction unique.

— Focus visible partout.

— Modales conformes.

— Boutons-icônes nommés.

— Rien n’est coupé sur un écran de 320 px.

Le piège de ce lot

La région d’annonce doit être unique dans le document. Deux régions live produisent des annonces doubles, pires que pas d’annonce.

Lot 15 — Démarrage et hors-ligne

Vague 3 · 1 j 1/2

CONSTATS PE-01, PE-02, PE-03, PE-04, PE-05, PE-06

FICHIERS index.html, js/registre.js, js/boot.js

QUAND Après le lot 14.

PROMPT DU LOT 15 — À COLLER APRÈS LE PROMPT MAÎTRE

Lot 15 — Le site doit s'ouvrir vite, et fonctionner sans réseau.

Constat(s) : PE-01, PE-02, PE-03, PE-04, PE-05, PE-06.

But : sur un téléphone d'entrée de gamme en 4G, le planning doit s'afficher sans attendre, et rester consultable sans connexion.

Fais :

1. defer sur tous les scripts non critiques, et chargement à la demande des modules lourds (OCR, PDF, sons) au premier geste qui les utilise.

2. Un manifeste et un service worker qui mettent en cache la coquille et les fragments. Les données étant locales, le planning doit fonctionner hors ligne.

3. Un script en échec ne doit plus bloquer tout le site : isole les modules et affiche un message dans une zone de repli.

4. Ajoute un paramètre de version sur la troisième feuille de style, et inclus-le dans les vérificateurs pour qu'un oubli casse la CI.

5. Mesure avant et après : nombre de requêtes et poids au premier affichage. Donne-moi les deux chiffres.

Ce que je ne veux pas : changer l'architecture en fragments, ajouter un bundler, réécrire les feuilles de style.

Vous devez obtenir

— Moins de requêtes au premier affichage.

— Le planning s’ouvre sans réseau.

— Un échec de script est isolé.

— Chiffres avant / après fournis.

Le piège de ce lot

Un service worker mal versionné sert l’ancienne version pour toujours. Prévois une stratégie de mise à jour explicite, sinon tu ne verras plus tes propres correctifs.

Lot 16 — Contenu, promesses et inscription

Reste · 1 j 1/2

CONSTATS	CO-01, CO-02, CO-03, CO-04, CO-05, CO-06, CO-07, CO-08, CO-09, CO-10, AU-02, AU-03, AU-04, AU-05, AU-11, AU-13, AU-14, AU-15, AU-16
FICHIERS	pages/accueil.html, pages/onboarding-*.html, pages/compte.html, pages/aide.html, css/
QUAND	Peut se faire en parallèle du lot 15.

PROMPT DU LOT 16 — À COLLER APRÈS LE PROMPT MAÎTRE

Lot 16 — Le site doit dire la vérité, et l'inscription doit pouvoir se finir.

Constat(s) : CO-01 à CO-10, AU-02 à AU-05, AU-11, AU-13 à AU-16.

But : plus aucune promesse non tenue, et une inscription que l'on peut reprendre et sauter.

Fais :

1. Accueil : retire ou reformule les deux cartes qui annoncent des fonctions inexistantes. Passe tout au tutoiement, comme le reste du site. Rends non cliquable le compteur de stress.

2. Inscription : un bandeau « étape 4 sur 12 », un lien « Passer cette étape » sur les étapes non indispensables, une reprise possible après fermeture, et le focus sur le premier champ à chaque étape.

3. Export : la fonction existe et n'est appelée nulle part. Ajoute un bouton « Télécharger mes données », et un lien depuis la confirmation d'effacement. La confirmation d'effacement doit énumérer ce qui sera effacé.

4. Aide : cinq questions/réponses repliées en haut, e-mail facultatif quand l'élève est connecté.

5. Un lien partagé doit avoir un titre, une description et une image (og:). Un pied de page doit dire qui édite le site.

6. Une feuille d'impression qui garde le planning du jour en noir sur blanc, et un bouton « Imprimer ».

7. Corrige les titres de la page Légende (h3 sans h2), la page EE & IA vide, et le bouton « Publier » du tutoriel qui ne dit pas pourquoi il est bloqué.

8. Page Soutien : un état vide explicite, une zone d'annonce sur la zone de réponse, et type="button" sur les quinze boutons qui n'en ont pas.

Ce que je ne veux pas : refondre l'accueil, changer la charte, ajouter une page.

Vous devez obtenir

— Aucune promesse non tenue sur l’accueil.

— Inscription sautable et reprenable.

— Export accessible.

— Impression correcte.

— La page Soutien n’est plus blanche quand elle est vide.

Le piège de ce lot

Si tu réécris l’accueil, ne touche pas aux cartes qui décrivent des fonctions réelles : elles servent de mode d’emploi.

Lot 17 — Dette technique

Reste · 3 j

CONSTATS AR-01, AR-02, AR-03, AR-04, AR-05

FICHIERS js/ (transversal)

QUAND En dernier, ou avant tout gros chantier.

PROMPT DU LOT 17 — À COLLER APRÈS LE PROMPT MAÎTRE

Lot 17 — Enlever les pièges du code avant qu'ils ne se referment.

Constat(s) : AR-01, AR-02, AR-03, AR-04, AR-05.

But : qu'un correctif appliqué au bon endroit ait réellement un effet, et qu'une erreur laisse une trace.

Fais, un point par session si nécessaire :

1. Vingt fonctions sont définies dans plusieurs fichiers (generateDayEvents, renderPlanning, openEditModal, saveEditedEvent, deleteEditedEvent, chain, fixed...). La dernière chargée gagne. Dresse la liste, dis-moi laquelle est réellement utilisée, et propose un plan de fusion sans casser le comportement. Ne fusionne qu'après mon accord.

2. Ajoute un journal sur chaque try/catch muet, avec le contexte. Un échec de génération ne doit plus ressembler à une journée vide.

3. Encapsule les 31 setItem dans un utilitaire qui attrape l'erreur de quota, purge les données les plus anciennes et prévient une fois.

4. Supprime le module pomodoro.js et son entrée au registre (déjà fait au lot 12 si tu l'as enchaîné).

5. Regroupe les sept clés du minuteur dans un objet unique et versionné, avec migration.

Ce que je ne veux pas : de refonte de l'architecture, de découpage en modules, de changement des noms publics utilisés par les pages.

Vous devez obtenir

— Plus aucune erreur avalée sans trace.

— Quota plein géré proprement.

— Inventaire des doublons validé.

— Un seul stockage pour le minuteur.

Le piège de ce lot

Le point 1 touche le moteur de planning. Ne fusionne rien sans avoir exécuté les vérificateurs sur les deux versions.

Lot 18 — Nouvelles capacités

Optionnel · 3 à 5 j

CONSTATS ET-09, ET-10, AU-08, AC-08

FICHIERS js/pages/emploi-temps.js, js/pages/planning-moteur.js, js/pages/exercices.js, css/

QUAND Seulement si vous les voulez. Ce ne sont pas des bugs.

PROMPT DU LOT 18 — À COLLER APRÈS LE PROMPT MAÎTRE

Lot 18 — Quatre ajouts, à faire un par un, seulement si tu les valides.

Ce ne sont pas des corrections : ce sont des fonctions qui manquent. Avant chacune, dis-moi ce que ça change pour l'élève et ce que ça coûte.

1. Un cours annulé pour une date précise : une table d'exceptions par date, un bouton « Ce cours n'a pas eu lieu » sur le bloc du planning du jour, et un repli des heures creuses. La semaine suivante ne doit pas être touchée. Purge après 60 jours.

2. Semaine A / semaine B : un champ parité sur chaque cours, et un filtre basé sur le numéro de semaine. Les cours existants sont « toutes les semaines ».

3. Le fichier d'un exercice est jeté après lecture. Stocke-le en IndexedDB avec une limite explicite, ou dis clairement que seul le nom est conservé.

4. Un thème sombre, en variables CSS, pour les trois écrans les plus utilisés.

Pour chacun : pose-moi les questions nécessaires avant d'écrire la première ligne, et n'implémente rien tant que je n'ai pas répondu.

Vous devez obtenir

— Une décision explicite par point, avant tout code.

— Aucun impact sur les fonctions existantes.

Le piège de ce lot

ET-10 (semaine A/B) touche le moteur de planning : c’est le changement le plus risqué de toute la liste, et le moins urgent.

Les cinq contrôles

À utiliser entre les lots, ou juste après. Ce sont eux qui font la différence entre un correctif et une intention.

Relire un diff avant de l’accepter

Juste après un lot, avant de committer.

PROMPT DE CONTRÔLE

Relis le diff que tu viens de produire, comme si tu ne l'avais pas écrit.

Cherche, dans cet ordre :
1. Ce que tu as cassé ailleurs : une fonction renommée encore appelée, un identifiant HTML supprimé encore lu, un style qui ne s'applique plus.
2. Ce que tu as changé sans que je l'aie demandé.
3. Les cas limites que tu n'as pas traités : valeur vide, null, date passée, tableau vide, double clic.
4. Les erreurs avalées : un try/catch sans message, une promesse sans catch.

Pour chaque point trouvé, donne le fichier, la ligne, et la correction. Ne corrige rien avant de me l'avoir montré.

Vérifier avant de livrer

Avant chaque commit.

PROMPT DE CONTRÔLE

Avant de considérer ce lot terminé, donne-moi :

1. La liste des fichiers modifiés, avec le nombre de lignes ajoutées et supprimées.
2. Le résultat de node --check sur chaque fichier JavaScript modifié.
3. Le résultat de chaque vérificateur, un par un (node tools/verifier-*.js), en indiquant ceux qui échouent et pourquoi.
4. La manipulation exacte que je dois faire pour constater le correctif, étape par étape, en partant d'un état propre.
5. Ce qui n'est pas couvert par ce lot, et qui reste à faire.

Si un vérificateur échoue, ne le contourne pas : dis-moi ce qu'il détecte et propose une correction.

Tester la voix et le son sur un vrai navigateur

Lots 13 et 12 (minuteur).

PROMPT DE CONTRÔLE

Le correctif touche au son. Un test hors du site ne compte pas.

Donne-moi un scénario de test à faire moi-même, dans l'ordre exact :
1. Le fichier à ouvrir, le geste à faire, ce que je dois entendre.
2. Le cas où la musique doit se taire complètement avant que la voix parle.
3. Le cas où je relance la pause pendant l'annonce (le son ne doit pas se superposer).
4. Le cas où je change d'onglet pendant la pause (le son doit continuer ou s'arrêter, mais pas se dédoubler).

Précise pour chaque étape ce qui doit se passer et ce qui signalerait un défaut. Ne me dis pas que c'est bon sans que je te l'aie confirmé.

Écrire un test de non-régression

Quand un bug est corrigé et pourrait revenir.

PROMPT DE CONTRÔLE

Ce défaut pourrait revenir à la prochaine modification. Écris un test de non-régression qui échoue sur l'ancien comportement et passe sur le nouveau.

Contraintes : le test doit tourner avec les outils déjà présents dans tools/ (pas de nouvelle dépendance), s'exécuter en moins de dix secondes, et ne rien envoyer sur le réseau. Donne-moi la commande pour le lancer, et le résultat avant et après correction.

Faire le point sur ce qui reste

À la fin de chaque vague.

PROMPT DE CONTRÔLE

Fais le point sur l'audit, sans rien corriger.

Pour les constats suivants : <liste d'identifiants>, dis-moi pour chacun :
1. Corrigé, avec la preuve (fichier et ligne du correctif).
2. Partiellement corrigé, et ce qui manque.
3. Non commencé.

Puis liste les constats qui, d'après ce que tu as vu dans le code, demandent une décision de ma part avant d'être corrigés — et pose-moi les questions une par une.

Ce qu’il ne faut pas écrire

Six demandes qui semblent raisonnables et qui produisent, sur ce dépôt précis, un site cassé ou un travail impossible à vérifier.

« Corrige tous les problèmes de l’audit »

L’assistant va modifier cinquante fichiers, en oublier la moitié, casser trois fonctions et vous annoncer que tout est fait. Vous ne pourrez ni vérifier, ni revenir en arrière proprement.

À la place : Un lot par session, avec les identifiants concernés.

« Fais en sorte que ce soit mieux »

« Mieux » n’est pas un critère. Vous obtiendrez une refonte de l’interface et un nouveau bug sur le chrono, alors que vous vouliez trois lignes de confirmation.

À la place : Décrivez le comportement observable : « quand je supprime un cours, un toast de 8 secondes propose Annuler ».

« Optimise le code »

Sans mesure, l’optimisation est une opinion. Et sur ce dépôt, elle touche le moteur de planning, qui est la partie la plus fragile.

À la place : « Le planning met 900 ms à s’afficher sur mon téléphone ; trouve pourquoi et corrige uniquement ça. »

« Refais la page comme tu veux »

Vous perdez la main sur un produit qui a des règles métier précises — et l’assistant n’a aucune raison de les connaître.

À la place : Décrivez écran par écran, et gardez les lots de design séparés des lots de comportement.

« Ajoute des tests partout »

Cent tests qui échouent pour des raisons de configuration ne valent pas un test qui protège le bug que vous venez de corriger.

À la place : « Écris le test qui empêche CE bug de revenir », un par lot.

« Ignore les erreurs de la console »

Une erreur JavaScript non traitée, sur ce site, signifie souvent une fonction qui ne s’exécute pas du tout. C’est exactement ce que l’audit reproche déjà au code (AR-02).

À la place : « Corrige l’erreur affichée dans la console, et dis-moi ce qui l’a causée. »

Tableau de suivi

À imprimer ou à garder ouvert. Ce qui n’est pas coché n’est pas fait, même si l’assistant a dit qu’il l’avait fait. La dernière colonne sert à noter la date et le commit.

	Lot	Constats	Effort	Commit / date
☐	0. Fermer la porte	SE-01, SE-02, SE-03, SE-04, SE-05	6 h	
☐	1. L’onglet Examens qui ne répond pas	DE-01, DE-02, DE-21, DE-22	1 h 30	
☐	2. Le planning ne ment plus sur le temps	DE-14, DE-15, DE-16, DE-19	3 h 30	
☐	3. La page Séance montre le bon jour, et garde le décor	PO-01, PO-02	1 h 30	
☐	4. Supprimer sans perdre	ET-02, ET-03, DE-05, AU-07, AU-10	2 h 30	
☐	5. Les exercices ont enfin un état	AU-06, AU-09	1 h 30	
☐	6. Voir la semaine prochaine	AU-01, PO-19	1 j	
☐	7. Lire une photo d’emploi du temps	ET-05, ET-06, ET-07, ET-08, ET-15	5 h	
☐	8. Corriger l’emploi du temps sans le casser	ET-01, ET-04, ET-11, ET-12, ET-13, ET-14, ET-17, ET-18	6 h	
☐	9. Synchronisation : ne plus perdre de travail	SY-01, SY-02, SY-03, SY-04, DE-25, ET-16	8 h	
☐	10. Modifier un devoir, modifier un examen	DE-03, DE-04, DE-06, DE-07	4 h 30	
☐	11. Finir la page Devoirs	DE-08, DE-09, DE-10, DE-11, DE-12, DE-13, DE-17, DE-18, DE-20, DE-23, DE-24, DE-26	5 h	
☐	12. Le minuteur qui tient	PO-03, PO-04, PO-05, PO-11, PO-12, PO-20	5 h	
☐	13. Le son et la voix	PO-06, PO-07, PO-08, PO-09, PO-10, PO-13, PO-14, PO-15, PO-16, PO-17, PO-18	6 h	
☐	14. Le clavier, les modales, les contrastes	AC-01, AC-02, AC-03, AC-04, AC-05, AC-06, AC-07, AC-09, AC-10, AC-11, AU-12	8 h	
☐	15. Démarrage et hors-ligne	PE-01, PE-02, PE-03, PE-04, PE-05, PE-06	1 j 1/2	
☐	16. Contenu, promesses et inscription	CO-01, CO-02, CO-03, CO-04, CO-05, CO-06, CO-07, CO-08, CO-09, CO-10, AU-02, AU-03, AU-04, AU-05, AU-11, AU-13, AU-14, AU-15, AU-16	1 j 1/2	
☐	17. Dette technique	AR-01, AR-02, AR-03, AR-04, AR-05	3 j	
☐	18. Nouvelles capacités	ET-09, ET-10, AU-08, AC-08	3 à 5 j	

Quand vous aurez fini

Reprenez le PDF d’audit et parcourez l’annexe : chaque identifiant coché ici devrait correspondre à un constat fermé. Les constats qui resteront ouverts sont, en pratique, ceux qui demandaient une décision de produit — le lot 18, et les questions du chapitre « Questions avant d’agir » de l’audit.